GESTIÓN DE GIMNASIO ALWAYS-STRONG

MARCO ANTONIO CANUX RAQUEC

GRUPO “EL LLANERO SOLITARIO”

EDUIN SALAS - (TRAINER COL)
ANDRE LOPEZ - (TRAINER GUA)



CAMPUSLANDS
SALON QUETZAL 1
RUTA NODEJS
GUATEMALA, GUATEMALA
2026
GESTIÓN DE GIMNASIO

1. SITUACIÓN PROBLEMA

El gimnasio AlwayStrong actualmente no cuenta con ningún sistema digital que le permita gestionar de una manera eficiente la información de sus clientes, servicios y operaciones internas. La mayoría de los procesos aún se realizan de forma manual o incluso sin una estructura centralizada, lo cual genera dificultades en la gestión, seguimiento y toma de decisiones.

En la situación actual, en el gimnasio StrongTech:

- No existe un sistema unificado para el correcto registro de clientes.
- La información de inscripciones de los clientes se maneja de forma desorganizada.
- No se cuenta con un control automatizado sobre los servicios ofrecidos.
- No se lleva un seguimiento estructurado del progreso de los clientes.
- La asignación de clientes a planes de entrenamiento se realiza sin validaciones automáticas.
- No existen reportes claros para evaluar el rendimiento o los riesgos de los clientes.
- No existen reportes financieros que evalúen el rendimiento del gimnasio.

El objetivo identificado para este proyecto es desarrollar una aplicación de línea de comandos (CLI) utilizando las herramientas de Node.js y MySQL, incluyendo programación orientada a objetos (POO), aplicación de principios SOLID e implementación de patrones de diseño. Todo esto para que la aplicación final permita, ya sea a un entrenador personal o gimnasio, gestionar de forma integral a sus clientes, los planes de entrenamiento que ofrece entre sus servicios, el seguimiento del progreso de los clientes, la alimentación personalizada y la gestión financiera. Esta herramienta debe permitir registrar contratos, asociar rutinas con fechas límite, registrar avances y llevar control financiero (ingresos por mensualidades, egresos por servicios asociados, etc.).

2. LEVANTAMIENTO DE REQUERIMIENTOS

El levantamiento de requerimientos se realizó mediante el análisis del funcionamiento actual del gimnasio, identificando sus necesidades operativas, los sujetos involucrados y los procesos críticos del negocio.

Se definieron los principales módulos del sistema (clientes, planes de entrenamiento, seguimiento físico, nutrición, contratos y gestión financiera), así como las funcionalidades necesarias para garantizar una gestión eficiente y segura.

campuslands

3. REQUERIMIENTOS

3.1. Requerimientos Funcionales

- Gestión de clientes:
  - RF-01: Crear, listar, actualizar y eliminar clientes.
  - RF-02: Asociar clientes a uno o varios planes de entrenamiento.
- Gestión de planes de entrenamiento:
  - RF-03: Crear planes con nombre, duración, metas físicas, nivel.
  - RF-04: Asociar a uno o varios clientes.
  - RF-05: Registrar contrato al asignar el plan a un cliente.
  - RF-06: Permitir renovar, cancelar o finalizar un plan.
- Seguimiento físico:
  - RF-07: Registrar avances semanales: peso, grasa corporal, medidas, fotos, comentarios.
  - RF-08: Visualizar progreso cronológico.
  - RF-09: Permitir eliminar registros.
- Nutrición:
  - RF-10: Crear planes de alimentación asociados al cliente y al plan de entrenamiento.
  - RF-11: Registrar alimentos por día.
  - RF-12: Consultar reporte nutricional semanal.
- Contratos:
  - RF-13: Contrato generado automáticamente al asignar un plan.
  - RF-14: Campos: condiciones, duración, precio, fecha inicio/fin.
  - RF-15: Asociado al cliente y plan correspondiente.
- Gestión financiera:
  - RF-16: Registro de ingresos (mensualidades, sesiones individuales).
  - RF-17: Registro de egresos (servicios, suplementos, gastos operativos).
  - RF-18: Consulta de balance financiero por fecha o por cliente.
  - RF-19: Implementación de transacciones para evitar inconsistencias en pagos.

campuslands

3.2. Requerimientos No Funcionales

- **RNF-01 — Rendimiento**:
  Las operaciones CRUD (clientes, planes, seguimiento) deben responder en menos de 2 segundos con un volumen de hasta 500 registros por tabla.

- **RNF-02 — Usabilidad:**
  La interfaz de consola debe emplear inquirer para menús y formularios interactivos, y chalk para diferenciar mensajes de éxito (verde), advertencia (amarillo) y error (rojo).

- **RNF-03 — Seguridad:**
  Las credenciales de conexión a MySQL y cualquier dato sensible deben manejarse mediante variables de entorno (dotenv) y el archivo .env nunca debe subirse al repositorio.

- **RNF-04 — Confiabilidad / Consistencia**
  Toda operación crítica (generación de contratos, pagos, cancelaciones) debe ejecutarse dentro de una transacción real de MySQL (BEGIN/COMMIT/ROLLBACK) para garantizar atomicidad.

- **RNF-05 — Mantenibilidad**
  El código debe organizarse en capas independientes (/models, /services, /commands, /config, /utils), seguir principios SOLID y aplicar al menos dos patrones de diseño, de forma que agregar una funcionalidad no requiera modificar módulos no relacionados.

- **RNF-06 — Portabilidad**
  La aplicación debe ejecutarse en cualquier sistema operativo con Node.js 18 o superior y acceso a un servidor MySQL 8.x, sin dependencias específicas de plataforma.

- **RNF-07 — Compatibilidad**
  3 Gestión de Always-Strong — Planeación SCRUM
  La persistencia de datos debe implementarse exclusivamente con el driver oficial mysql2, evitando ORMs que oculten el manejo explícito de transacciones.

- **RNF-08 — Documentación**
  El repositorio debe incluir un README.md completo, un archivo .gitignore adecuado para Node.js, y un historial de commits que siga el estándar Conventional Commits.

- **RNF-09 — Disponibilidad de datos**
  Toda la información debe persistir permanentemente en MySQL; el cierre inesperado de la aplicación no debe provocar pérdida de datos ya confirmados (commits).

campuslands

4. HISTORIAS DE USUARIO

HISTORIA DE USUARIO HU-01			
Prioridad: Alta			
CÓDIGO DEL REQUERIMIENTO:	RF-01	**Actor**	Entrenador/ Administrador
NOMBRE DEL REQUERIMIENTO	**Registro del cliente**		
Descripción			
Como entrenador puedo registrar un nuevo cliente con sus datos personales y de contacto con el fin de llevar un control centralizado de todas las personas atendidas en el gimnasio.			
Funcionalidad			
Permitir capturar por consola (usando inquirer) los datos del cliente (nombre, edad, contacto, fecha de ingreso) validando cada campo antes de persistirlo en MySQL mediante el repositorio de clientes.			
Criterios de aceptación	1. El sistema debe rechazar el registro si falta un campo obligatorio. 2. El sistema debe validar formato de correo/teléfono. 3. El sistema debe confirmar con chalk (verde) el registro exitoso.….		
Restricciones			
No se puede registrar un cliente con un identificador (DPI/correo) ya existente en la base de datos.			

HISTORIA DE USUARIO HU-02			
Prioridad: Alta			
CÓDIGO DEL REQUERIMIENTO:	RF-01	**Actor**	Entrenador/ Administrador
NOMBRE DEL REQUERIMIENTO	**Actualización y eliminación de cliente**		
Descripción			
Como entrenador puedo listar, actualizar o eliminar un cliente existente con el fin de mantener la información siempre correcta y depurada.			
Funcionalidad			
Mostrar un listado de clientes desde MySQL, permitir seleccionar uno para editar sus campos o eliminarlo, validando que la operación no rompa relaciones activas (planes/contratos vigentes).			
Criterios de aceptación	1. El sistema debe listar los clientes ordenados por fecha de ingreso. 2. El sistema debe advertir si el cliente tiene planes activos antes de eliminar. 3. La actualización debe re-ejecutar las validaciones de campo.		
Restricciones			
No se puede eliminar un cliente con contratos activos sin antes finalizarlos o cancelarlos.			

HISTORIA DE USUARIO HU-03			
Prioridad: Alta			
CÓDIGO DEL REQUERIMIENTO:	RF-02	**Actor**	Entrenador/ Administrador
NOMBRE DEL REQUERIMIENTO	**Asociación de cliente a plan de entrenamiento**		
Descripción			
Como entrenador puedo asociar uno o varios planes de entrenamiento a un cliente con el fin de formalizar el servicio que recibirá.			
Funcionalidad			
Permitir seleccionar un cliente existente y uno o varios planes disponibles, creando la relación cliente-plan y disparando la generación automática del contrato.			
Criterios de aceptación	1. El sistema debe impedir asociar un plan inexistente o inactivo. 2. El sistema debe permitir múltiples planes simultáneos por cliente. 3. Cada asociación debe generar automáticamente un contrato (RF-05).		
Restricciones			
Un mismo plan no puede asociarse dos veces activo al mismo cliente simultáneamente.			

HISTORIA DE USUARIO HU-04			
Prioridad: Alta			
CÓDIGO DEL REQUERIMIENTO:	RF-03/RF-04	**Actor**	Entrenador/ Administrador
NOMBRE DEL REQUERIMIENTO	**Creación de plan de entrenamiento**		
Descripción			
Como entrenador puedo crear un plan de entrenamiento con nombre, duración, metas físicas y nivel con el fin de ofrecerlo como servicio a mis clientes.			
Funcionalidad			
Capturar y validar los datos del plan (nombre, duración en semanas, metas, nivel: principiante/intermedio/avanzado) y persistirlo para que pueda asociarse a uno o varios clientes.			
Criterios de aceptación	1. El nivel debe pertenecer únicamente a los tres valores permitidos. 2. La duración debe ser un número entero positivo. 3. El plan creado queda disponible de inmediato para asociación.		
Restricciones			
No se permite crear un plan con nombre duplicado exacto.			

HISTORIA DE USUARIO HU-05			
Prioridad: Alta			
CÓDIGO DEL REQUERIMIENTO:	RF-03/ RF-13/ RF-14/ RF-15	**Actor**	Entrenador/ Administrador
NOMBRE DEL REQUERIMIENTO	**Generación automática de contrato**		
Descripción			
Como entrenador quiero que el sistema genere automáticamente un contrato al asignar un plan a un cliente con el fin de formalizar condiciones, precio y vigencia sin pasos manuales adicionales.			
Funcionalidad			
Al confirmarse la asociación cliente-plan, el sistema debe crear un registro de contrato (condiciones, duración, precio, fecha inicio/fin) dentro de la misma transacción que crea la asociación, usando el patrón Factory para construir el contrato.			
Criterios de aceptación	1. El contrato debe crearse en la misma transacción que la asociación cliente-plan. 2. Si la creación del contrato falla, la asociación debe revertirse (rollback). 3. El contrato debe quedar vinculado al cliente y al plan correspondiente.		
Restricciones			
No puede existir una asociación cliente-plan activa sin un contrato asociado.			

HISTORIA DE USUARIO HU-06			
Prioridad: Alta			
CÓDIGO DEL REQUERIMIENTO:	RF-06	**Actor**	Entrenador/ Administrador
NOMBRE DEL REQUERIMIENTO	**Renovación, cancelación o finalización de plan**		
Descripción			
Como entrenador puedo renovar, cancelar o finalizar el plan de un cliente con el fin de reflejar el estado real del servicio contratado.			
Funcionalidad			
Permitir cambiar el estado del plan del cliente (activo, renovado, cancelado, finalizado). La cancelación debe revertir (rollback) el seguimiento físico y el contrato asociados, dentro de una transacción real de MySQL.			
Criterios de aceptación	1. Al cancelar, el sistema debe hacer rollback del contrato y seguimientos relacionados si aplica. 2. Al renovar, se debe generar un nuevo periodo de contrato. 3. Todo cambio de estado debe quedar registrado con fecha.		
Restricciones			
No se puede renovar un plan que ya fue cancelado; debe crearse una nueva asociación.			

HISTORIA DE USUARIO HU-07			
Prioridad: Media			
CÓDIGO DEL REQUERIMIENTO:	RF-07	**Actor**	Entrenador/ Administrador
NOMBRE DEL REQUERIMIENTO	**Registro de avance semanal**		
Descripción			
Como entrenador puedo registrar el avance semanal de un cliente (peso, grasa corporal, medidas, enlaces a fotos, comentarios) con el fin de dar seguimiento objetivo a su progreso.			
Funcionalidad			
Capturar por consola los datos de seguimiento asociados a un cliente y a su plan activo, validando rangos numéricos razonables y guardando enlaces de imagen como texto.			
Criterios de aceptación	1. El peso y la grasa corporal deben ser valores numéricos dentro de rangos válidos. 2. El enlace de foto debe tener formato de URL válido. 3. El registro debe quedar asociado a la fecha de la semana correspondiente.		
Restricciones			
No se puede registrar un avance para un cliente sin un plan activo.			

HISTORIA DE USUARIO HU-08			
Prioridad: Media			
CÓDIGO DEL REQUERIMIENTO:	RF-08/RF-09	**Actor**	Entrenador/ Administrador
NOMBRE DEL REQUERIMIENTO	**Visualización y eliminación de progreso**		
Descripción			
Como entrenador puedo visualizar el progreso cronológico de un cliente y eliminar registros incorrectos con el fin de mantener un historial confiable.			
Funcionalidad			
Listar los registros de seguimiento ordenados por fecha en formato de tabla en consola, y permitir eliminar un registro puntual con rollback si su eliminación afecta la consistencia del plan.			
Criterios de aceptación	1. El listado debe mostrarse en orden cronológico ascendente. 2. La eliminación debe pedir confirmación previa. 3. Si la eliminación afecta reportes ya generados, se debe aplicar rollback.		
Restricciones			
No se puede eliminar el único registro de seguimiento de un contrato activo sin advertencia explícita.			

HISTORIA DE USUARIO HU-09			
Prioridad: Media			
CÓDIGO DEL REQUERIMIENTO:	RF-10/RF-11	**Actor**	Entrenador/ Administrador
NOMBRE DEL REQUERIMIENTO	**Creación de plan de alimentación y registro de alimentos**		
Descripción			
Como entrenador puedo crear un plan de alimentación asociado a un cliente y su plan de entrenamiento, y registrar alimentos por día con calorías estimadas, con el fin de personalizar la nutrición del cliente.			
Funcionalidad			
Permitir crear un plan nutricional vinculado a cliente y plan de entrenamiento, y registrar alimentos día a día con su valor calórico estimado, acumulando totales diarios.			
Criterios de aceptación	1. Cada alimento debe tener un valor calórico numérico mayor a cero. 2. El plan nutricional debe estar vinculado a un cliente con plan de entrenamiento activo. 3. El sistema debe calcular el total calórico diario automáticamente.		
Restricciones			
No se puede registrar alimentos en días fuera del rango de vigencia del plan de entrenamiento.			

HISTORIA DE USUARIO HU-10			
Prioridad: Media			
CÓDIGO DEL REQUERIMIENTO:	RF-12	**Actor**	Entrenador/ Administrador
NOMBRE DEL REQUERIMIENTO	**Reporte nutricional semanal**		
Descripción			
Como entrenador puedo consultar el reporte nutricional semanal de un cliente con el fin de evaluar si está cumpliendo su plan de alimentación.			
Funcionalidad			
Agregar por semana los registros diarios de alimentos del cliente y presentar en consola un resumen de calorías totales y promedio diario.			
Criterios de aceptación	1. El reporte debe agrupar correctamente los días dentro de la semana seleccionada. 2. Debe mostrar el promedio calórico diario. 3. Debe indicar si no existen registros para la semana consultada.		
Restricciones			
El reporte solo puede generarse para semanas dentro del periodo de vigencia del plan nutricional.			

HISTORIA DE USUARIO HU-11			
Prioridad: Alta			
CÓDIGO DEL REQUERIMIENTO:	RF-16/RF-17/ RF-19	**Actor**	Entrenador/ Administrador
NOMBRE DEL REQUERIMIENTO	**Registro de ingresos y egresos con transacciones**		
Descripción			
Como entrenador puedo registrar ingresos (mensualidades, sesiones) y egresos (servicios, suplementos, gastos operativos) con el fin de llevar el control financiero del negocio de forma consistente.			
Funcionalidad			
Registrar movimientos financieros dentro de una transacción real de MySQL (BEGIN/COMMIT/ROLLBACK), asegurando que un fallo a mitad de la operación no deje registros parciales o inconsistentes.			
Criterios de aceptación	1. Todo registro financiero debe ejecutarse dentro de una transacción. 2. Si ocurre un error durante el registro, se debe revertir automáticamente. 3. El monto debe ser numérico y mayor a cero.		
Restricciones			
No se puede registrar un ingreso asociado a un contrato inexistente o cancelado.			

HISTORIA DE USUARIO HU-12			
Prioridad: Media			
CÓDIGO DEL REQUERIMIENTO:	RF-18	**Actor**	Entrenador/ Administrador
NOMBRE DEL REQUERIMIENTO	**Consulta de balance financiero**		
Descripción			
Como entrenador puedo consultar el balance financiero por fecha o por cliente con el fin de conocer el estado económico del gimnasio.			
Funcionalidad			
Calcular la diferencia entre ingresos y egresos filtrando por rango de fechas o por cliente específico, mostrando el resultado en consola con chalk (positivo en verde, negativo en rojo).			
Criterios de aceptación	1. El filtro por fecha debe aceptar rango inicio-fin. 2. El filtro por cliente debe mostrar solo sus movimientos asociados. 3. El balance debe mostrarse con formato de moneda.		
Restricciones			
Si no existen movimientos en el rango o cliente consultado, se debe informar balance en cero, no un error.			

5. METODOLOGÍA

El proyecto se desarrollará bajo el marco de trabajo SCRUM, adaptado a un euipo de trabajo de un único integrante. Dado que Always-Strong es un proyecto individual, Marco Antonio Canux Raquec asume simultáneamente los tres roles definidos por SCRUM, manteniéndolos separados en cuanto a responsabilidades y momentos del proceso:

- Product Owner: define y prioriza el backlog (las historias de usuario), decide qué se entrega en cada sprint y valida el resultado contra los requerimientos.
- Scrum Master: vela por el cumplimiento del marco de trabajo, documenta impedimentos y facilita las ceremonias (planning, revisión, retrospective).
- Developer; implementa las historias de usuario siguiendo POO, SOLID y los patrones de diseño seleccionados (Repository y Factory).

Herramienta de seguimiento: ClickUp, donde se documentará el backlog completo, los sprints, las historias de usuario con sus criterios de aceptación, prioridad y estado.
Debido a que el proyecto completo deberá entregarse entre el 22/09/2026 y el 28/09/2026, se definieron los siguientes sprints cortos.

5.1 Sprint 0 – Planeación (22/09/2026)

Objetivo: cerrar la documentación de planeación SCRUM (roles, backlog, historias de usuario, requerimientos) y dejar organizado el tablero en ClickUp con todas las tareas del Sprint 1 y Sprint 2.

5.2 Sprint 1 – Núcleo del sistema (23/09/2026 – 25/09/2026)

Objetivo: dejar funcional la base técnica del proyecto y los módulos de clientes, planes y contratos.

- Configuración del repositorio, .gitignore, variables de entorno y conexión a MySQL.
- Diseño del esquema de base de datos y creación de tablas.
- Implementación de /models con validaciones por campo.
- Patrón Repository para el acceso a datos.
- CRUD de clientes (HU-01, HU-02) y planes (HU-04).
- Asociación cliente-plan y generación automática de contrato con patrón Factory (HU-03, HU-05).
- Renovación/cancelación/finalización de plan con rollback (HU-06).

5.3 Sprint 2 – Seguimiento, nutrición, finanzas y cierre (26/09/2026 – 28/09/2026)

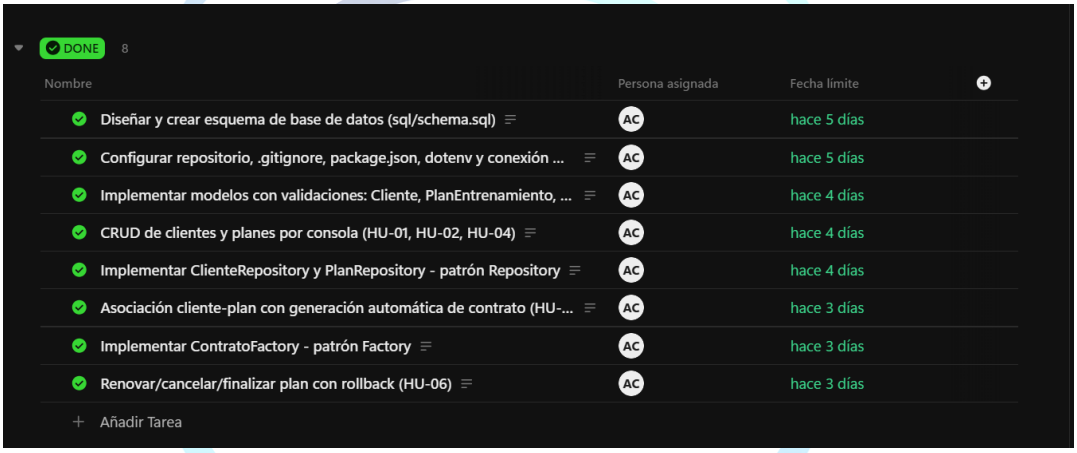
Objetivo: completar los módulos restantes, la capa financiera con transacciones reales, y todos los entregables documentales.

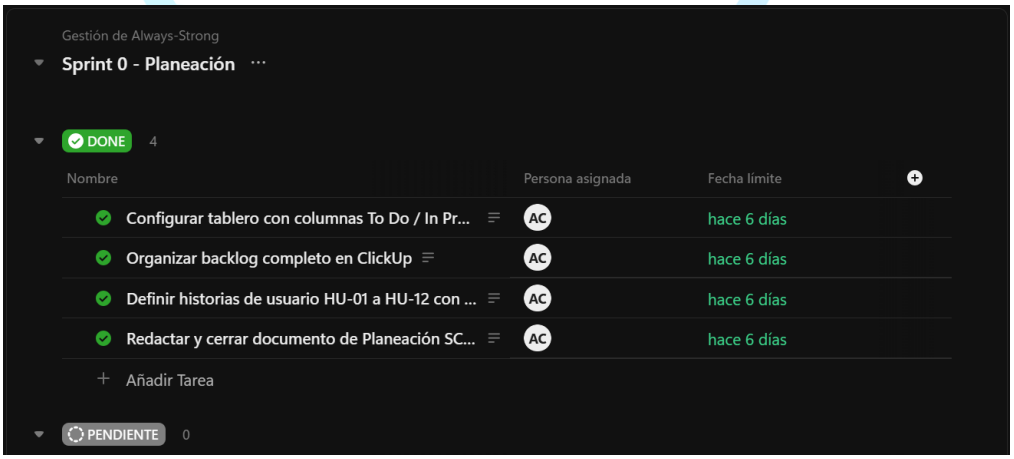
- Seguimiento físico y visualización cronológica con rollback (HU-07, HU-08).
- Módulo de nutrición y reporte semanal (HU-09, HU-10).

- Gestión financiera con transacciones reales (BEGIN/COMMIT/ROLLBACK) e informes de balance (HU-11, HU-12).
- Redacción final de README.md, revisión de Conventional Commits y .gitignore.
- Grabación del video de presentación (máx. 7 minutos).
- Cierre de evidencias en ClickUp y GitHub, y llenado de la sección de Evidencia y Conclusiones de este documento.

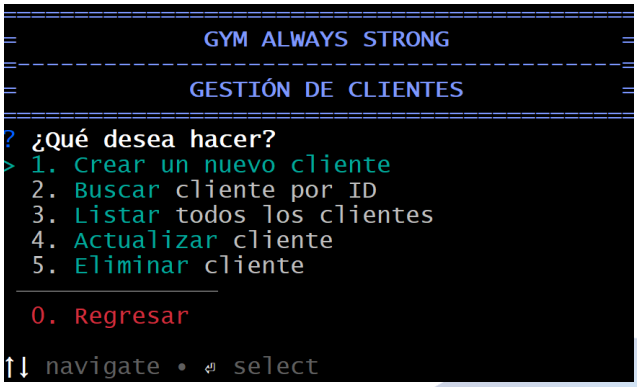
6. EVIDENCIA DE PLANTEAMIENTO DE PLATAFORMA DE TRABAJO

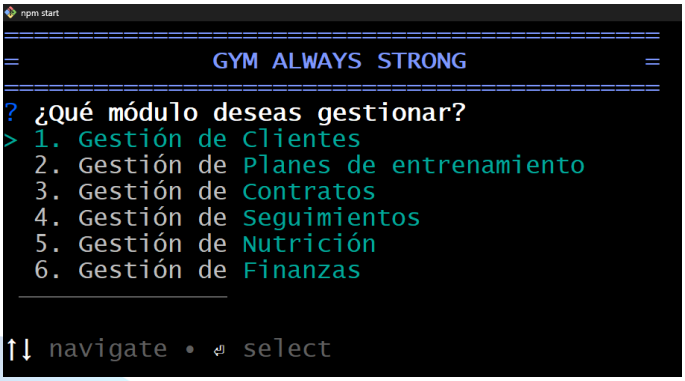
- Link del repositorio trabajado:
  - https://github.com/canuxantonio502/alwaysstrong_nodejs_campuslands
- Evidencia del tablero Scrum:



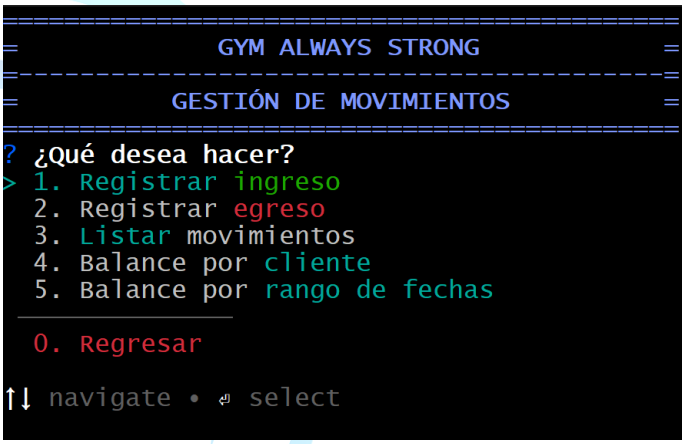


- **Funcionamiento del producto final:**









campuslands

- **7. CONCLUSIONES**

- **Conclusiones generales:** El desarrollo del sistema CLI para Always-Strong permitió estructurar exitosamente la gestión del gimnasio mediante Node.js y MySQL. La implementación de una arquitectura en capas combinada con los patrones Repository y Factory aseguró la mantenibilidad y desacoplamiento del código. Asimismo, el uso estricto de transacciones en la base de datos garantiza la integridad de la información financiera, de contratos y del seguimiento físico ante posibles fallos en la ejecución.

- **Conclusiones de la retrospectiva del Sprint:** La adopción del marco Scrum, ajustado a los roles asumidos de manera individual, facilitó el cumplimiento paulatino de los requerimientos y el control del alcance en los plazos establecidos. Como oportunidad de mejora se identificó la conveniencia de integrar pruebas automatizadas en futuros sprints para validar las transacciones críticas de forma previa a la integración final.

campuslands